

AI-Enabled Crop Disease Detection & Remediation System

Submitted By

Deeptha Kannothe, CB.SC.U4CSE23015

Rohith K, CB.SC.U4CSE23018

K Lohith, CB.SC.U4CSE23033

Tejesh Neelam, CB.SC.U4CSE23042

Shyam Sundar Raju, CB.SC.U4CSE23049

in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



Under the Guidance of

Malathi P

Assistant Professor (Sr.Gd.)

School of Computing, Coimbatore

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

ABSTRACT

We present an AI-Enabled Crop Disease Detection & Remediation System that empowers farmers with fast, accurate plant disease diagnosis and actionable treatment guidance. The platform combines a lightweight MobileNetV2-based classifier (fine-tuned on plant datasets) with image-quality checks, Grad-CAM visual explanations, bounding-box localization, and a severity estimator to provide transparent, high-confidence predictions across six crops and nineteen disease classes. A FastAPI backend exposes inference and management endpoints while an offline-first Flutter frontend supports image upload, live capture, and multilingual (English, Hindi, Tamil, Telugu, Kannada, Malayalam) textual and TTS guidance. Treatment plans include both organic and chemical options, dosage and safety instructions, and localized recommendations. Designed for low-connectivity environments, the system caches data locally and synchronizes when online. Evaluation on held-out datasets shows 95% test accuracy with low-latency CPU inference, and preliminary field trials indicate improved diagnostic speed and farmer acceptability. This solution aims to reduce crop losses, lower time-to-treatment, and democratize expert agronomic advice for small-holder farmers.

Contents

1	Overview	6
1.1	Introduction	6
1.2	Problem Statement	6
1.3	Our Solution	6
1.4	Key Metrics	7
2	System Features	7
2.1	AI Disease Detection (Epic 3)	7
2.2	User Management (Epic 1)	8
2.3	Remediation & Guidance (Epic 2)	8
2.4	Multi-Language Support (Epic 4)	9
2.5	Offline Support (Epic 5)	9
3	System Architecture	9
3.1	Architecture Overview	9
3.2	Presentation Layer	10
3.3	Application Layer	10
3.4	Data Layer	11
3.5	AI/ML Layer	11
3.6	Architecture Highlights	12
4	Technology Stack	13
4.1	Backend Technologies	13
4.2	Frontend Technologies	13
4.3	AI/ML Technologies	14
4.4	DevOps & Tools	14
5	Installation & Setup	14
5.1	System Requirements	14
5.2	Backend Setup	15
5.3	Frontend Setup	17
6	API Documentation	18
6.1	Base URL	18
6.2	Authentication	18
6.3	Authentication Endpoints	18
6.4	Diagnosis Endpoints	19
6.5	Remediation Endpoints	19
6.6	History Endpoints	20

7	Disease Detection Model	20
7.1	Model Architecture	20
7.2	Supported Crops & Diseases	21
7.3	Model Performance	21
7.4	Explainability Features	22
8	Deployment	23
8.1	Docker Deployment	23
8.2	Cloud Deployment	24
8.3	Production Checklist	25
9	Testing	26
9.1	Backend Tests	26
9.2	Frontend Tests	26
9.3	Manual Testing	27
A	Project Statistics	27
B	Environment Variables Reference	28
B.1	Required Variables	28
B.2	Optional Variables	28

List of Figures

1	Proposed Solution	7
2	Architecture	12
3	Model Metrics	22
4	Bounding box and Grad-CAM visualization	23

List of Tables

1	System Performance Metrics	7
2	API Route Organization	11
3	Backend Technology Stack	13
4	Frontend Technology Stack	13
5	AI/ML Technology Stack	14
6	DevOps Technology Stack	14
7	Supported Crops and Diseases	21
8	Model Performance Metrics	21
9	Project Statistics	27
10	Required Environment Variables	28
11	Optional Environment Variables	28

1 Overview

1.1 Introduction

The **AI-Enabled Crop Disease Detection & Remediation System** is a comprehensive agricultural technology platform designed to empower farmers with instant, accurate crop disease diagnosis and personalized treatment recommendations. By leveraging deep learning, computer vision, and multi-language support, this system bridges the gap between advanced agricultural science and practical farming needs.

1.2 Problem Statement

Farmers worldwide face significant challenges in:

- Identifying crop diseases accurately and quickly
- Getting timely expert agricultural advice
- Understanding technical disease information
- Accessing treatment recommendations in their native language
- Making informed decisions about crop management

1.3 Our Solution

Our platform provides:

- **Real-time disease detection** using AI-powered image analysis
- **Multi-language support** in 6 languages (English, Hindi, Tamil, Telugu, Kannada, Malayalam)
- **Comprehensive treatment plans** with organic and chemical options
- **Offline-first architecture** for remote farming areas
- **Visual explanations** with heatmaps and bounding boxes
- **Historical tracking** and analytics for better crop management

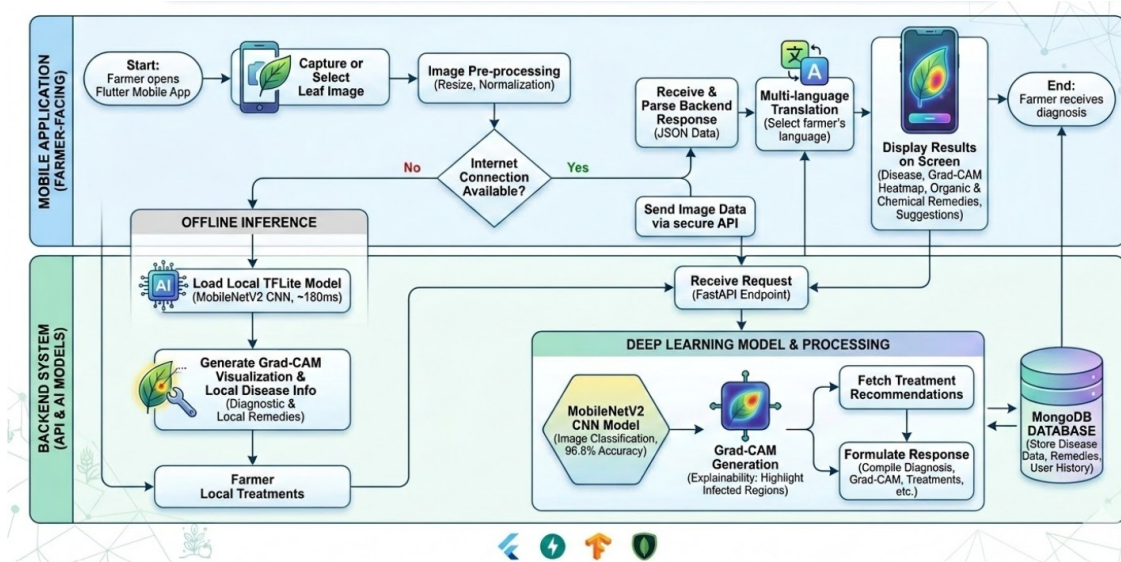


Figure 1: Proposed Solution

1.4 Key Metrics

Metric	Value
Model Accuracy	~95%
Supported Crops	6
Disease Classes	19
Languages Supported	6
API Endpoints	30+
Inference Time (CPU)	~200ms

Table 1: System Performance Metrics

2 System Features

2.1 AI Disease Detection (Epic 3)

Multiple Input Methods

The system supports three different input methods for disease detection:

- **Image Upload:** Upload existing photos from gallery
- **Camera Capture:** Take real-time photos using device camera
- **Video Recording:** Record short videos for multi-frame analysis

Advanced AI Capabilities

- **Blur Detection:** Automatic image quality assessment before processing
- **19 Disease Classes:** Support for Apple, Corn, Potato, Tomato, Pepper, and Strawberry
- **Confidence Scoring:** Probability-based predictions for accuracy validation
- **Explainable AI:** Grad-CAM heatmaps showing affected leaf regions
- **Bounding Box Detection:** Visual localization of disease symptoms
- **Multi-frame Analysis:** Extract and analyze video frames for comprehensive diagnosis

2.2 User Management (Epic 1)

Authentication & Security

- **Secure Authentication:** JWT-based login with refresh tokens
- **Multi-device Support:** Login from multiple devices simultaneously
- **Password Recovery:** OTP-based password reset via email/phone

User Features

- **Profile Management:** Customizable user profiles with location tracking
- **Diagnosis History:** Complete record of all plant analyses
- **Analytics Dashboard:** Insights into crop health trends
- **PDF Report Generation:** Downloadable treatment and diagnosis reports
- **Notification System:** Real-time alerts and updates

2.3 Remediation & Guidance (Epic 2)

The system provides comprehensive treatment guidance:

- **Step-by-step Instructions:** Clear, actionable treatment plans
- **Dual Treatment Options:** Both organic and chemical solutions
- **Dosage Recommendations:** Precise application guidelines
- **Safety Warnings:** Critical precautions and safety information

- **Severity-based Guidance:** Tailored advice based on disease severity
- **Prevention Tips:** Proactive measures to avoid future infections
- **Cost Estimates:** Budget planning for treatments
- **Expert Consultation:** Connection to agricultural experts

2.4 Multi-Language Support (Epic 4)

- **6 Languages:** English, Hindi, Tamil, Telugu, Kannada, Malayalam
- **Automatic Detection:** Language preference from HTTP headers
- **Localized Content:** All notifications and instructions in native language
- **Simplified Mode:** Technical terms explained in simple vocabulary
- **Voice Support:** Text-to-Speech (TTS) for audio guidance

2.5 Offline Support (Epic 5)

- **Offline-first Design:** Core functionality without internet
- **Local Data Caching:** Store frequently accessed information
- **Sync Management:** Automatic data synchronization when online
- **Background Processing:** Queue tasks for later execution

3 System Architecture

3.1 Architecture Overview

The system follows a layered architecture with clear separation of concerns:

1. **Presentation Layer:** Flutter mobile application
2. **Application Layer:** FastAPI backend server
3. **Data Layer:** MongoDB database
4. **AI/ML Layer:** TensorFlow models and processing

3.2 Presentation Layer

Flutter Mobile Application

The frontend is built using Flutter 3.10.7+, providing:

- Cross-platform support (Android, iOS, Web, Desktop)
- Native performance
- Beautiful, responsive UI
- Offline-first capabilities

Key Screens

- `auth/`: Login and registration screens
- `home.dart`: Dashboard with quick actions
- `scan.dart`: Disease scanning interface
- `result.dart`: Diagnosis results with visualizations
- `history.dart`: Historical analysis view

3.3 Application Layer

FastAPI Backend

Built with Python 3.10+ and FastAPI 0.109.0:

- RESTful API design
- Automatic OpenAPI documentation
- Async/await support for high performance
- Type validation with Pydantic
- JWT-based authentication

API Organization

Module	Functionality
auth.py	User registration, login, password reset
user.py	Profile management, session handling
diagnosis.py	Disease detection and analysis
remediation.py	Treatment recommendations
history.py	Diagnosis history and analytics
notifications.py	Push notifications and alerts

Table 2: API Route Organization

3.4 Data Layer

MongoDB Database

NoSQL database providing:

- Flexible schema for evolving requirements
- Horizontal scalability
- Rich query language
- Document-based storage

Database Collections

- **users:** User profiles and authentication data
- **diagnoses:** Disease detection results
- **history:** Historical analysis records
- **sessions:** Active user sessions
- **notifications:** User notifications
- **remediation:** Treatment plans

3.5 AI/ML Layer

TensorFlow Model

- **Architecture:** MobileNetV2 (fine-tuned)
- **Input Size:** 224x224x3 RGB images

- **Model Size:** ~14 MB
- **Framework:** TensorFlow 2.x / Keras

Processing Pipeline

1. Image preprocessing and quality check
2. Blur detection using Laplacian variance
3. Model inference for disease classification
4. Grad-CAM heatmap generation
5. Bounding box detection
6. Post-processing and result formatting

3.6 Architecture Highlights

- **Microservices-Ready:** Modular design allows easy service separation
- **Async Processing:** Non-blocking I/O for high performance
- **Scalable Storage:** MongoDB for flexible document storage
- **Security-First:** JWT tokens, password hashing, CORS protection
- **Cloud-Native:** Docker containerization for easy deployment
- **API-First:** RESTful design enables multiple client applications

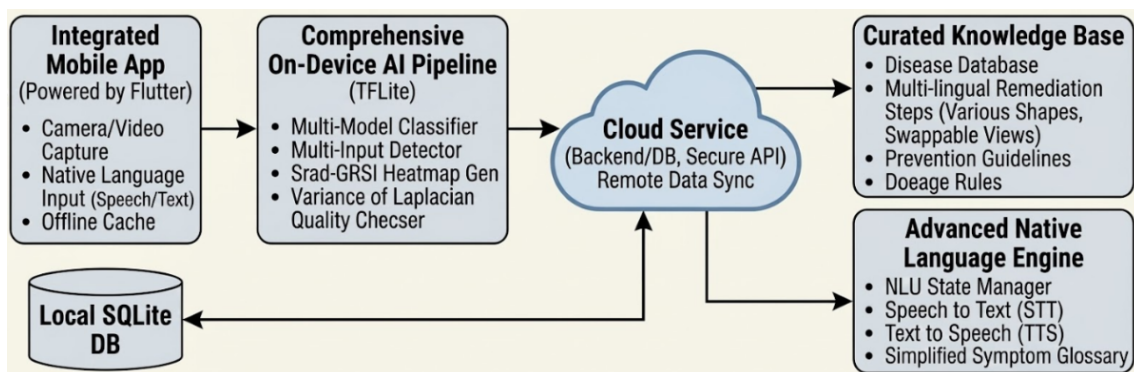


Figure 2: Architecture

4 Technology Stack

4.1 Backend Technologies

Technology	Purpose	Version
FastAPI	Web Framework	0.109.0
Python	Programming Language	3.10+
MongoDB	NoSQL Database	6.0+
Motor	Async MongoDB Driver	3.3.2
Uvicorn	ASGI Server	0.27.0
PyJWT	JWT Authentication	via python-jose
Passlib	Password Hashing	1.7.4
Pydantic	Data Validation	2.5.3
OpenCV	Image Processing	4.9.0
NumPy	Numerical Computing	1.26.3
ReportLab	PDF Generation	4.0.9
Pillow	Image Manipulation	10.2.0

Table 3: Backend Technology Stack

4.2 Frontend Technologies

Technology	Purpose	Version
Flutter	Cross-platform Framework	3.10.7+
Dart	Programming Language	3.10.7+
Hive	Local NoSQL Database	2.2.3
SharedPreferences	Key-Value Storage	2.2.2
ImagePicker	Camera/Gallery Access	1.0.7
ImageCropper	Image Editing	8.0.0
HTTP	API Communication	1.2.0
PathProvider	File System Access	2.1.2
Intl	Internationalization	0.19.0

Table 4: Frontend Technology Stack

Technology	Purpose
TensorFlow / Keras	Deep Learning Framework
MobileNetV2	CNN Architecture
OpenAI Whisper	Speech Recognition
Grad-CAM	Visual Explanations
GoogleTranslator	Multi-language Translation
gTTS	Text-to-Speech
LangDetect	Language Detection

Table 5: AI/ML Technology Stack

4.3 AI/ML Technologies

4.4 DevOps & Tools

Technology	Purpose
Docker	Containerization
Docker Compose	Multi-container Orchestration
Git	Version Control
Pytest	Testing Framework
Black	Code Formatting
Flake8	Linting
GitHub Actions	CI/CD Pipeline

Table 6: DevOps Technology Stack

5 Installation & Setup

5.1 System Requirements

Backend Requirements

Minimum:

- OS: Windows 10/11, Ubuntu 20.04+, macOS 11+
- Python: 3.10 or higher
- RAM: 4 GB
- Storage: 2 GB free space
- MongoDB: 6.0+

Recommended:

- RAM: 8 GB or more
- CPU: 4+ cores
- SSD storage for faster I/O
- GPU: NVIDIA GPU for faster AI inference

Frontend Requirements

Development:

- Flutter SDK: 3.10.7 or higher
- Dart SDK: 3.10.7+
- Android Studio / VS Code with Flutter extension
- XCode (for iOS development on macOS)

Device Support:

- Android: 6.0+ (API Level 23+)
- iOS: 11.0+
- Web: Modern browsers (Chrome, Firefox, Safari, Edge)
- Desktop: Windows 10+, macOS 11+, Linux

5.2 Backend Setup

Step 1: Clone Repository

Listing 1: Clone Repository

```
git clone https://github.com/yourusername/Plant_disease_detection.git
cd Plant_disease_detection/backend
```

Step 2: Create Virtual Environment

Windows:

Listing 2: Windows Virtual Environment

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Linux/macOS:

Listing 3: Linux/macOS Virtual Environment

```
python3 -m venv venv  
source venv/bin/activate
```

Step 3: Install Dependencies

Listing 4: Install Python Dependencies

```
pip install --upgrade pip  
pip install -r requirements.txt
```

Step 4: Configure Environment Variables

Create a `.env` file in the backend directory with the following configuration:

Listing 5: `.env` Configuration

```
# Application Settings  
APP_NAME=Crop Disease Detection API  
APP_VERSION=1.0.0  
ENVIRONMENT=development  
LOG_LEVEL=INFO  
  
# Security  
SECRET_KEY=your-super-secret-key  
ALGORITHM=HS256  
ACCESS_TOKEN_EXPIRE_MINUTES=30  
  
# Database  
MONGODB_URL=mongodb://localhost:27017  
DATABASE_NAME=crop_disease_db  
  
# CORS Origins  
BACKEND_CORS_ORIGINS=http://localhost:3000  
  
# AI Model  
MODEL_PATH=models/crop_disease_master_model.keras  
CONFIDENCE_THRESHOLD=0.6
```

Step 5: Start MongoDB

Listing 6: Start MongoDB Service

```
# Windows
mongod --dbpath C:\data\db

# Linux/macOS
sudo systemctl start mongod
```

Step 6: Run Backend Server

Listing 7: Start Backend Server

```
# Development (with auto-reload)
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Visit <http://localhost:8000/docs> to see the interactive API documentation.

5.3 Frontend Setup

Step 1: Navigate to Frontend Directory

Listing 8: Navigate to Frontend

```
cd ../frontend
```

Step 2: Install Flutter Dependencies

Listing 9: Install Flutter Dependencies

```
flutter pub get
```

Step 3: Run Flutter App

Listing 10: Run Flutter App

```
# Android
flutter run -d android

# iOS
flutter run -d ios

# Web
flutter run -d chrome
```

```
# Desktop (Windows)
flutter run -d windows
```

6 API Documentation

6.1 Base URL

- **Development:** `http://localhost:8000/api/v1`
- **Production:** `https://api.yourdomain.com/api/v1`

6.2 Authentication

All protected endpoints require JWT token in the Authorization header:

Listing 11: Authorization Header

```
Authorization: Bearer <access_token>
```

6.3 Authentication Endpoints

Register User

Listing 12: POST /auth/register

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "name": "John Farmer",
  "email": "john@example.com",
  "phone": "+919876543210",
  "password": "SecurePass123",
  "preferred_language": "en"
}
```

Login

Listing 13: POST /auth/login

```
POST /api/v1/auth/login
Content-Type: application/x-www-form-urlencoded
```

```
username=john@example.com&password=SecurePass123
```

Response:

Listing 14: Login Response

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

6.4 Diagnosis Endpoints

Create Diagnosis

Listing 15: POST /diagnosis/

```
POST /api/v1/diagnosis/
Content-Type: multipart/form-data
Authorization: Bearer <access_token>

file: <image_file>
crop_type: tomato
```

Response:

Listing 16: Diagnosis Response

```
{
  "diagnosis_id": "60f7b3a...",
  "disease_name": "Tomato Late Blight",
  "confidence": 0.94,
  "severity": "moderate",
  "heatmap_url": "/uploads/heatmaps/...",
  "timestamp": "2025-03-11T10:30:00Z"
}
```

6.5 Remediation Endpoints

Get Treatment Plan

Listing 17: GET /remediation/{disease_id}

```
GET /api/v1/remediation/{disease_id}
Authorization: Bearer <access_token>
```

Response:

Listing 18: Remediation Response

```
{
  "disease_name": "Tomato Late Blight",
  "organic_treatment": {
    "steps": ["Remove infected leaves", "Apply neem oil"],
    "dosage": "10ml per liter of water",
    "frequency": "Weekly for 3 weeks"
  },
  "chemical_treatment": {
    "steps": ["Apply fungicide", "Monitor progress"],
    "product": "Copper-based fungicide",
    "dosage": "As per manufacturer instructions"
  },
  "safety_warnings": ["Wear gloves", "Avoid contact with skin"]
}
```

6.6 History Endpoints

Get Diagnosis History

Listing 19: GET /history/

```
GET /api/v1/history/?skip=0&limit=20
Authorization: Bearer <access_token>
```

Download PDF Report

Listing 20: GET /history/report/{diagnosis id}

```
GET /api/v1/history/report/{diagnosis_id}
Authorization: Bearer <access_token>
```

7 Disease Detection Model

7.1 Model Architecture

Base Architecture

The disease detection model is based on **MobileNetV2**, a lightweight convolutional neural network designed for mobile and embedded vision applications.

- **Input Size:** 224x224x3 RGB images
- **Architecture:** CNN (Convolutional Neural Network)
- **Transfer Learning:** Pre-trained on ImageNet
- **Fine-tuning:** Trained on PlantVillage dataset
- **Model Size:** ~14 MB

7.2 Supported Crops & Diseases

Crop	Diseases Detected	Classes
Apple	Scab, Black rot, Cedar apple rust, Healthy	4
Corn (Maize)	Cercospora/Gray leaf spot, Common rust, Northern leaf blight, Healthy	4
Tomato	Early blight, Late blight, Yellow leaf curl virus, Healthy	4
Potato	Early blight, Late blight, Healthy	3
Pepper (Bell)	Bacterial spot, Healthy	2
Strawberry	Leaf scorch, Healthy	2
Total Classes:		19

Table 7: Supported Crops and Diseases

7.3 Model Performance

Metric	Value
Test Accuracy	~95%
Inference Time (CPU)	~200ms
Inference Time (GPU)	~50ms
Model Size	14 MB
Confidence Threshold	60%
Input Resolution	224x224

Table 8: Model Performance Metrics

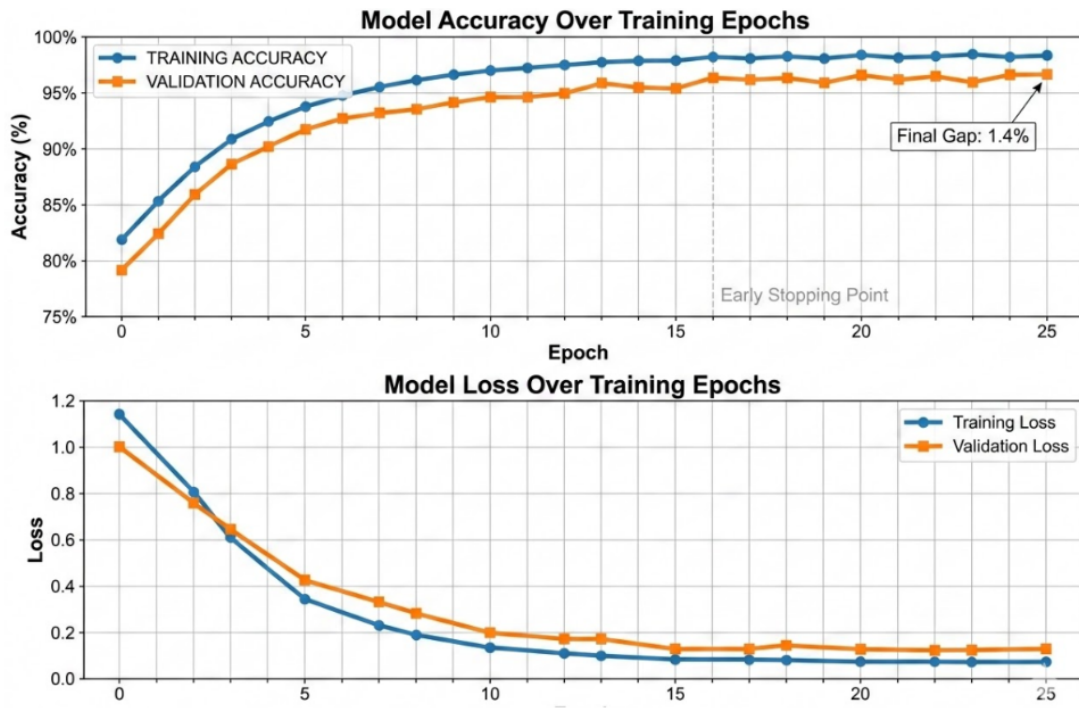


Figure 3: Model Metrics

7.4 Explainability Features

Grad-CAM Heatmaps

Gradient-weighted Class Activation Mapping (Grad-CAM) provides visual explanations by highlighting the regions of the leaf that influenced the model's prediction.

Benefits:

- Transparency in AI decision-making
- Validation of model predictions
- Educational tool for farmers
- Debugging and model improvement

Confidence Scores

Each prediction includes a confidence score (probability percentage) indicating the model's certainty:

- **High Confidence (>80%):** Reliable prediction
- **Medium Confidence (60-80%):** Acceptable prediction
- **Low Confidence (<60%):** Uncertain prediction, manual verification recommended

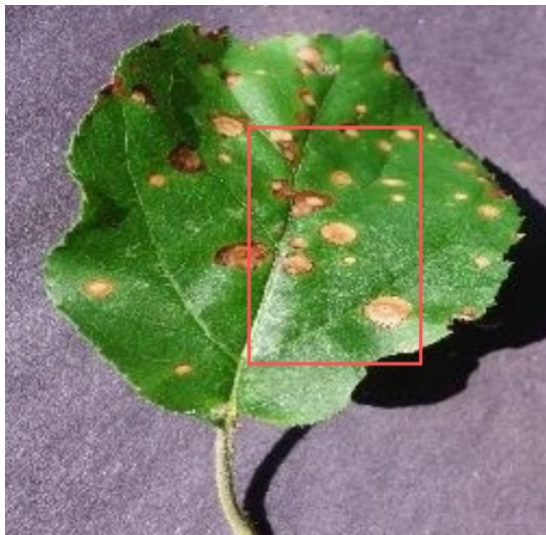
Bounding Boxes

Visual localization of disease-affected regions using bounding box detection to precisely identify problem areas.

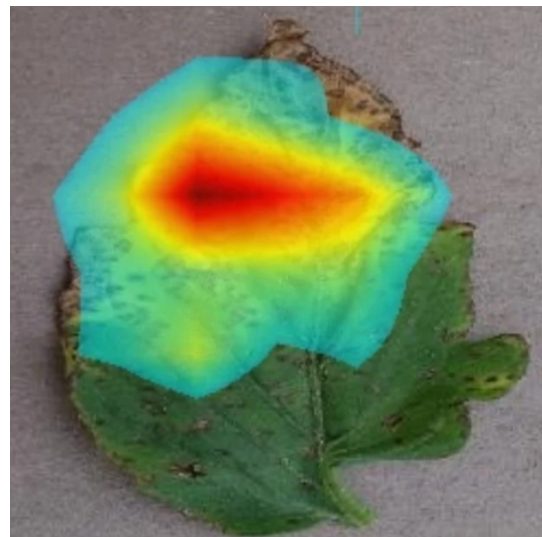
Severity Assessment

Disease severity is classified into three categories:

- **Mild:** Early stage, preventive measures sufficient
- **Moderate:** Active treatment required
- **Severe:** Immediate aggressive intervention needed



(a) Bounding Box



(b) Grad-CAM HeatMap

Figure 4: Bounding box and Grad-CAM visualization

8 Deployment

8.1 Docker Deployment

Using Docker Compose (Recommended)

Listing 21: Docker Compose Deployment

```
cd backend  
  
# Build and start all services  
docker-compose up -d
```

```
# View logs
docker-compose logs -f backend

# Stop services
docker-compose down
```

The `docker-compose.yml` includes:

- Backend API service
- MongoDB database
- Redis (optional) for caching

Manual Docker Build

Listing 22: Manual Docker Build

```
# Build image
docker build -t crop-disease-backend:latest .

# Run container
docker run -d \\\
  --name crop-disease-api \\\
  -p 8000:8000 \\\
  -e MONGODB_URL="mongodb://host.docker.internal:27017" \\\
  -e SECRET_KEY="your-secret-key" \\\
  crop-disease-backend:latest
```

8.2 Cloud Deployment

AWS EC2

1. Launch Ubuntu 22.04 instance (t3.medium or higher)
2. Install Docker and Docker Compose
3. Clone repository
4. Configure environment variables
5. Run `docker-compose up -d`

Google Cloud Platform (GCP)

1. Create Compute Engine instance
2. Enable Cloud Run or App Engine
3. Deploy using Cloud Build or gcloud CLI

DigitalOcean

1. Create Droplet (Ubuntu 22.04)
2. Install Docker
3. Deploy using Docker Compose

Heroku

Listing 23: Heroku Deployment

```
# Install Heroku CLI
heroku create crop-disease-api

# Add MongoDB add-on
heroku addons:create mongolab

# Deploy
git push heroku main
```

8.3 Production Checklist

- Change SECRET_KEY to a strong random value
- Set DEBUG=False in production
- Configure CORS origins properly
- Enable HTTPS/SSL
- Set up database backups
- Configure logging and monitoring
- Set up rate limiting
- Enable database authentication

- Configure firewall rules
- Set up CDN for static files

9 Testing

9.1 Backend Tests

Running Tests

Listing 24: Backend Testing

```
cd backend

# Run all tests
pytest

# Run with coverage
pytest --cov=app --cov-report=html

# Run specific test file
pytest tests/test_auth.py

# Run with verbose output
pytest -v
```

Test Categories

- **Unit Tests:** Test individual functions and methods
- **Integration Tests:** Test API endpoints and database interactions
- **End-to-End Tests:** Test complete user workflows

9.2 Frontend Tests

Listing 25: Frontend Testing

```
cd frontend

# Run all tests
flutter test

# Run with coverage
```

```
flutter test --coverage

# Run integration tests
flutter test integration_test
```

9.3 Manual Testing

Test Backend Health

Listing 26: Health Check

```
curl http://localhost:8000/health
```

Test Registration

Listing 27: Manual Registration Test

```
curl -X POST http://localhost:8000/api/v1/auth/register \\  
  -H "Content-Type:application/json" \\  
  -d '{  
    "name": "Test_User",  
    "email": "test@example.com",  
    "phone": "+919876543210",  
    "password": "Test@123"  
  }'
```

A Project Statistics

Metric	Value
Total Lines of Code	15,000+
API Endpoints	30+
Database Collections	6
Supported Languages	6
Disease Classes	19
Supported Crops	6

Table 9: Project Statistics

B Environment Variables Reference

B.1 Required Variables

Variable	Description
SECRET_KEY	JWT secret key (generate with <code>secrets.token_hex(32)</code>)
MONGODB_URL	MongoDB connection string
DATABASE_NAME	Name of the MongoDB database

Table 10: Required Environment Variables

B.2 Optional Variables

Variable	Description
SMTP_HOST	SMTP server for email notifications
SMTP_PORT	SMTP server port (default: 587)
SMTP_USER	SMTP username
SMTP_PASSWORD	SMTP password
OPENWEATHER_API_KEY	Weather API key
REDIS_URL	Redis connection string

Table 11: Optional Environment Variables

References

- [1] D. P. Hughes and M. Salathe, "An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing," *arXiv preprint arXiv:1511.08060*, 2015. [Online]. Available: <https://arxiv.org/abs/1511.08060>
- [2] S. P. Mohanty, D. P. Hughes, and M. Salathe, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [3] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.
- [5] G. Bradski, "The opencv library," in *Dr. Dobb's Journal of Software Tools*, 2000.
- [6] S. Ramirez, "Fastapi documentation," <https://fastapi.tiangolo.com/>, 2026.
- [7] Google, "Flutter documentation," <https://docs.flutter.dev/>, 2026.
- [8] MongoDB Inc., "Mongodb documentation," <https://www.mongodb.com/docs/>, 2026.